OpenTelemetry in the AI-based Village Pond Planning System

CSD Assignment 1 – Add-on: observability with traces, metrics and logs

RAHUL RAJ

STUDENT ID: 12341680, Indian Institute of Technology Bhilai, India

We added OpenTelemetry (OTel) to the existing Village Pond Planning System (Assignment 1). Every Python process — the FastAPI service and the two Celery worker pools — exports traces, metrics and logs over OTLP to an OpenTelemetry Collector. The collector routes traces to Jaeger, metrics to Prometheus and logs to Loki, and Grafana shows all three with links between them. Library instrumentation covers HTTP, SQL, Redis, the Celery broker and outbound provider calls. We also added one span per stage of the terrain pipeline (Priority-Flood, D8, site ranking, catchment), so a trace reads as the algorithm. The captured data answered two real questions about the running system. In a contour analysis, the hydrology algorithms take milliseconds and 52–70 % of the job is one network read. A pond design reported "confidence low" because a soil-database call had failed on DNS after 4 s; the trace and the warning log share one trace id, so the cause was found by following it.

Additional Key Words and Phrases: OpenTelemetry, Distributed Tracing, Observability, OTLP, Jaeger, Prometheus, Loki, Grafana

ACM Reference Format:

Rahul Raj, Student ID: 12341680. 2026. OpenTelemetry in the AI-based Village Pond Planning System: CSD Assignment 1 – Add-on: observability with traces, metrics and logs. 1, 1 (October 2026), 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 WHY OBSERVABILITY FOR THIS SYSTEM

The planner is a distributed system, even though it runs on one laptop. A request crosses the nginx front end, the FastAPI API, PostgreSQL, Redis (the Celery broker) and a Celery worker. The worker then reads remote data: the Copernicus DEM on AWS S3, Open-Meteo rainfall, the ISRIC SoilGrids soil database and OpenStreetMap geocoding. Before this add-on there were Prometheus counters and JSON logs with a request id, but no way to see one request's path across those processes. Nor could we see which part of a 2–8 s job was the algorithm and which part was waiting on a remote server.

OpenTelemetry is a vendor-neutral standard for three *signals*:

- **Traces.** A trace is a tree of *spans*. Each span is one timed operation with attributes, linked to its parent by a trace id and a span id. The *context* (those ids) travels with the work, across threads and across processes, in message headers.
- **Metrics.** Aggregated numbers over time: counters, histograms, gauges.
- **Logs.** Timestamped records that also carry the trace and span id of the code that wrote them.

Author's address: Rahul Raj
Student ID: 12341680, Indian Institute of Technology Bhilai, Bhilai, India, rahul.raj9237@gmail.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
Manuscript submitted to CSD Assignment

Every signal names its source with a *Resource*, for example `service.name = pond-worker-heavy`. Telemetry leaves the process over one wire protocol, OTLP.

2 IMPLEMENTATION

2.1 Architecture

Figure 1 shows the data path. The application knows only one address, the collector. Which backend stores which signal is collector configuration, so swapping Jaeger for Grafana Tempo, or adding a commercial backend, needs no code change.

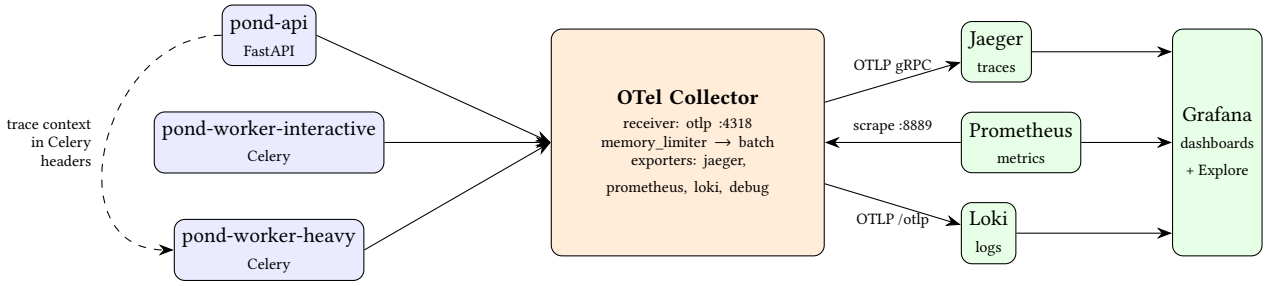


Fig. 1. Telemetry path. Each process pushes OTLP/HTTP to the collector; the collector fans signals out to one backend each; Grafana reads all three. Dashed: the trace context travels with the job through Redis, so the worker's spans join the API's trace.

2.2 What is instrumented

Table 1 lists every source of telemetry. Most of it is *automatic*: an instrumentation library patches a framework the app already uses. The domain-specific parts are *manual*.

Table 1. Sources of telemetry (`app/core/telemetry.py`)

Source	What it emits
FastAPI instrumentation	a server span per request (route, method, status); `http.server.request.duration` histogram
SQLAlchemy instrumentation	a span per SQL statement, with the statement text
Redis instrumentation	a span per Redis command (`LPUSH`, `LLEN`, ...)
Celery instrumentation	`apply_async` (publish) and `run` (execute) spans; injects and extracts the trace context in message headers
urllib instrumentation	a client span per outbound call: Open-Meteo, NASA POWER, SoilGrids, Nominatim
Job span (manual)	`job <kind>` around each analysis, with the job id, outcome and key results (`pond.catchment.area_ha`)
Stage spans (manual)	one child span per pipeline stage, e.g. `stage: filling sinks (Priority-Flood)`
Domain metrics (manual)	`pond.job.duration`, `pond.job.stage.duration` (s), `pond.jobs.active`, `pond.catchment.area` (ha), `pond.design.storage` (m^3)
OTLP log handler	every log record at INFO and above, with its trace and span id

2.3 Stage spans without touching the algorithms

The hydrology engines are pure numpy, and a test (`tests/test_layering.py`) fails the build if a framework is imported into them. Putting `tracer.start_span` inside them would break that rule. Every workflow already records its

progress with `jobs.update(stage="filling sinks (Priority-Flood)")` for the progress bar in the UI. `StageObservingJobs` is a Decorator around the job repository that turns each change of `stage` into a span:

```
def update(self, job_id, **fields):
    stages = _stage_spans.get() # set by job_span() for this job only
    stage = fields.get("stage")
    if stages is not None and isinstance(stage, str):
        if stage in {"done", "failed", "compensated"}:
            stages.close(error=...) # end the last stage span
        else:
            stages.enter(stage) # end previous, start + make current the next
    return self._inner.update(job_id, **fields)
```

Each stage span is made *current*, so the SQL and HTTP spans emitted during that stage nest under it. The engines import nothing new, so the layering test still passes.

2.4 Context propagation and log correlation

- **Across processes.** The Celery instrumentation writes the W3C `traceparent` into the task message; the worker reads it back, so its spans join the API's trace.
- **Across threads.** On the lab deployment, the thread-pool job runner submits each job inside `contextvars.copy_context()`, which carries the same context into the pool thread.
- **Logs.** The OTLP `LoggingHandler` sits on the root logger. It is re-attached after our logging setup and after Celery's, because both replace the root handlers. Every exported record carries its trace and span id. The JSON stdout formatter adds the same ids:

```
{"ts": "2026-10-16T07:19:37+0000", "level": "INFO", "logger": "uvicorn.access",
 "message": "172.21.0.1:61746 - \"GET /api/v1/jobs/ea6f4f0b-... HTTP/1.1\" 200",
 "trace_id": "f99435df7fb3cd30071f41000a09e5be", "span_id": "19522e380ff6f544"}
```

2.5 Tools, libraries and services

Table 2. Tools used

Component	Version and role
`opentelemetry-sdk`, `-api`	1.45.1 — TracerProvider, MeterProvider, LoggerProvider; batch processors
`opentelemetry-exporter-otlp-proto-http`	1.45.1 — OTLP/HTTP exporters for the three signals
`opentelemetry-instrumentation-*`	0.66b1 — fastapi, sqlalchemy, redis, celery, urllib, logging
OpenTelemetry Collector (contrib)	0.111.0 — receiver, processors, four exporters (`infra/otel/collector.yaml`)
Jaeger all-in-one	1.62.0 — trace storage and UI (:16686)
Prometheus	2.53.0 — scrapes the collector's metric endpoint
Grafana Loki	3.2.0 — log store with a native OTLP endpoint
Grafana	11.1.0 — dashboard *Pond — OpenTelemetry* and Explore; Loki↔Jaeger links

Running it. `make up` starts 14 containers, the three new ones being `otel-collector`, `jaeger` and `loki`. Telemetry is switched on by `POND_OTEL_ENABLED=true`, which is the Compose default. `make seed` then analyses the sample map. **Verification:** `tests/test_telemetry.py` uses in-memory exporters to assert five things:

(1) the stage spans are children of the job span;

(2) a failed job marks its stage and the job span as errors;
(3) JSON log lines carry the trace id;
(4) exported logs carry their trace context;
(5) one upload is one trace, from `POST /api/v1/analyzeContour` down to the Priority-Flood stage.

The full suite (226 tests), `ruff` and `mypy --strict` pass.

3 RESULTS: SCREENSHOTS FROM THE RUNNING SYSTEM

All screenshots below come from this project running locally under Docker Compose on 16 October 2026. They follow two user actions in the browser:

(1) analysing the provided sample contour map (`contours_1m.kml`, the Khapri area of Durg);
(2) clicking the top-ranked site and designing a pond there.

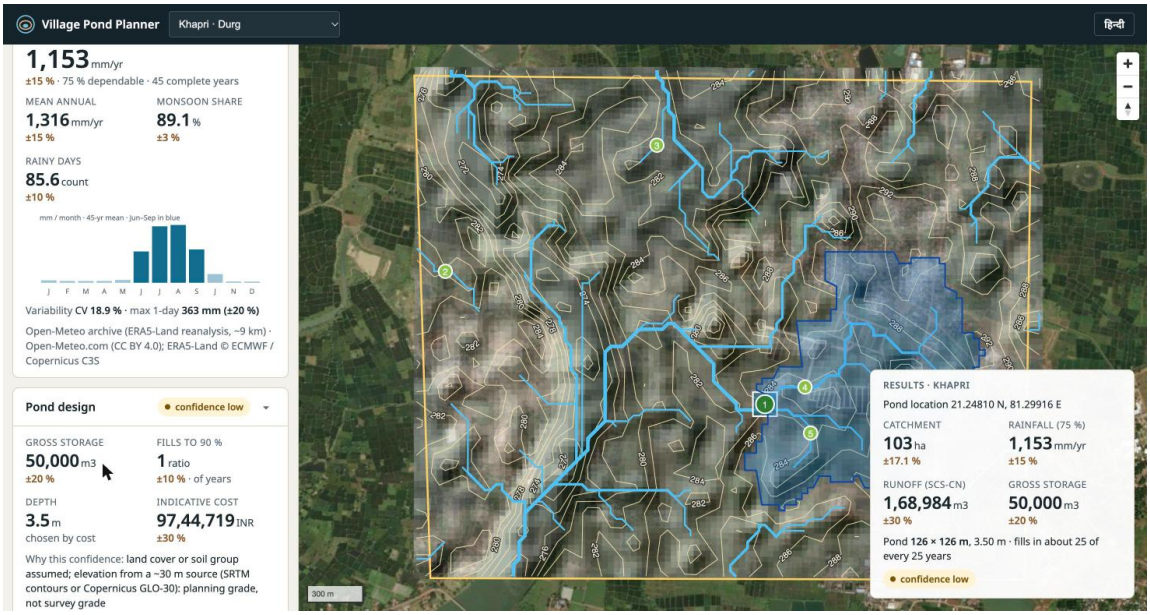


Fig. 2. The application being observed. Site #1 has a 103 ha catchment (blue). The pond design gives 50 000 m^3 and fills in about 25 of 25 years. The confidence badge reads "low", and Figure 5 explains why.

3.1 One trace across three processes

Figure 3 shows the trace for one upload, top to bottom:

(1) **API** (`pond-api`, teal): the `POST /api/v1/analyzeContour` server span. Under it are the SQL statements that create the job row (`INSERT pond`, `SELECT pond`) and a Redis `LLEN`, which is the backpressure check on queue depth.
(2) **Broker:** `apply_async/analysis.contour` with its `LPUSH`, which puts the job on the Redis queue.
(3) **Worker** (`pond-worker-heavy`, yellow): `run/analysis.contour` and our `job contour_analysis` span.

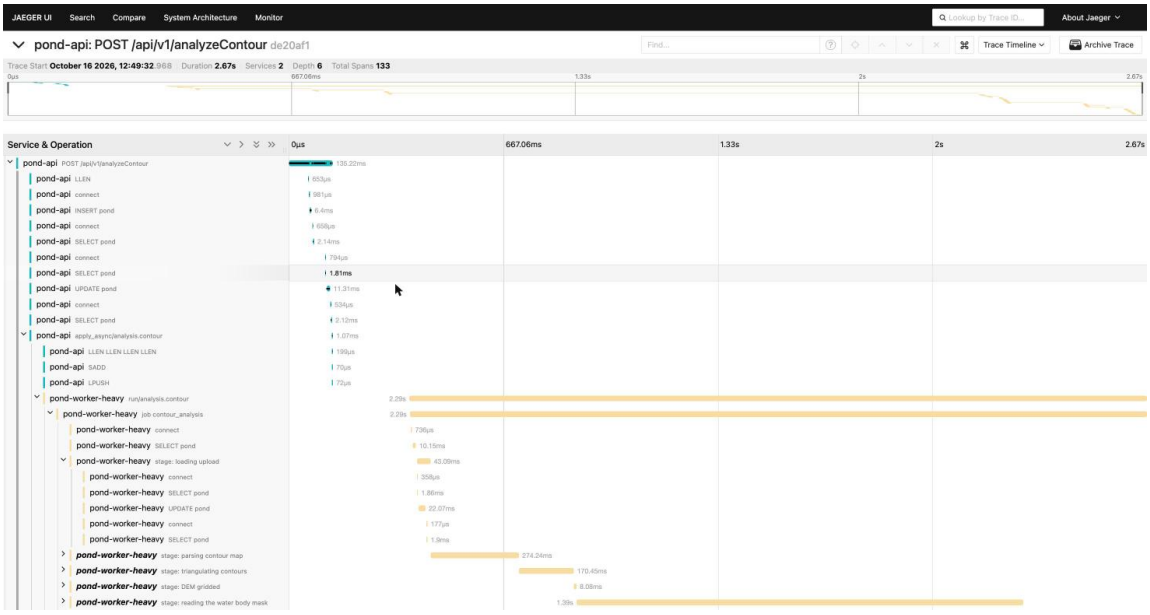


Fig. 3. Jaeger, trace `de20af1` (133 spans, 2 services, 2.67 s): one sample-map upload.

The worker spans are in the *same trace* as the HTTP request although they ran in another container: the context crossed Redis in the message headers. The gap between the API returning (135 ms) and the worker starting is the queueing delay, visible directly. Expanding `stage: loading upload` shows its SQL nested under it.

3.2 The algorithm as a waterfall

Figure 4 reads as the methodology section of the main report:

- parsing the KML (274 ms) and triangulating the contours (170 ms);
- reading the water-body mask (1.39 s);
- filling sinks with Priority-Flood (31 ms) and D8 flow routing with accumulation (8 ms);
- extracting streams (5 ms) and ranking pond sites (180 ms);
- persisting (about 110 ms) and delineating the top site's catchment (11 ms).

What this told us. The examined algorithms are not the bottleneck. Priority-Flood, D8 and catchment delineation together take about 50 ms. The single largest stage is *reading the water body mask*, a window read of a Copernicus GLO-30 file on AWS S3. It took 1.39 s of 2.67 s here (52 %) and 1.85 s of 2.66 s on the first run (70 %). So the place to optimise is this read, not the hydrology. Without stage spans we would have guessed the opposite.

The trace also shows the read's cache at work. The mask goes through an in-process `lru_cache` keyed by the grid. The run above was *cold*: it was the first job for that footprint in a freshly started worker. Re-analysing the same map later in the same worker (trace `0b7c157`, 24 October) took 5.8 ms for the same stage. The cache, however, lives in one process. A restarted worker, the other pool, or another lab replica pays the full 1.4–1.9 s again. The next optimisation the traces point to is a cache that all workers share (the warped mask in MinIO or Redis), or warming the cache when a worker starts.

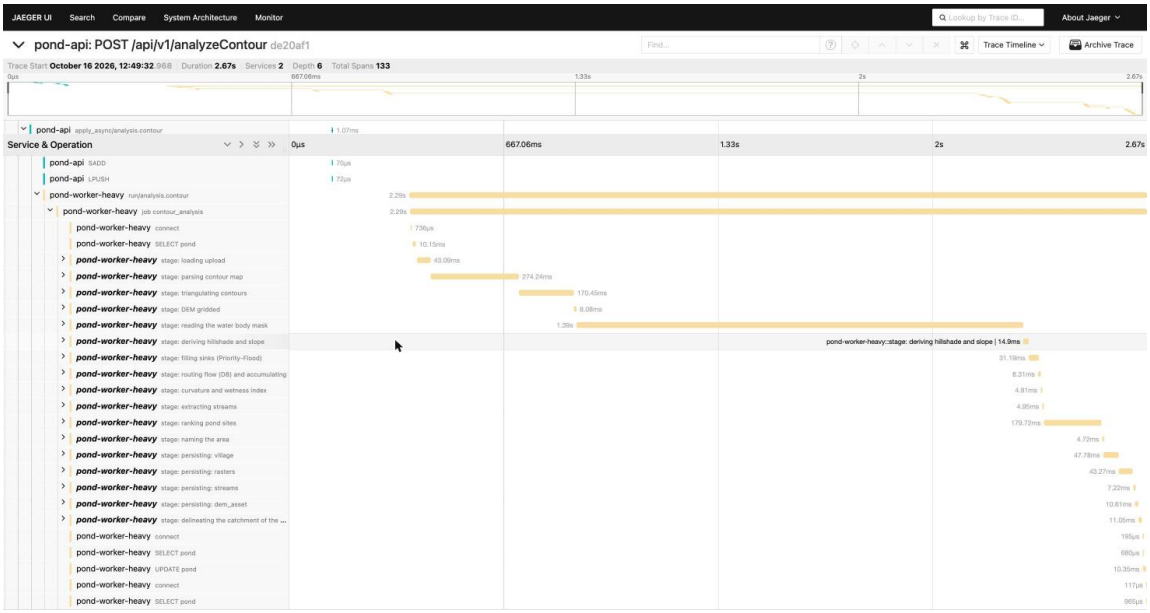


Fig. 4. The same trace, scrolled to the worker: one span per pipeline stage, in execution order.

3.3 Debugging a degraded result

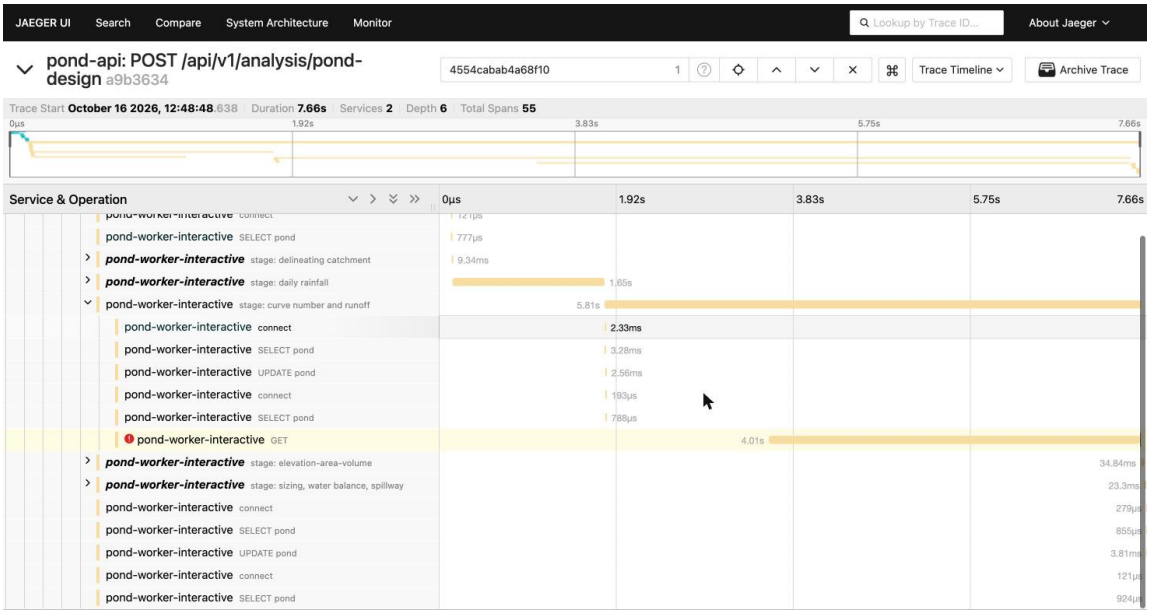


Fig. 5. Jaeger, trace a9b3634: the pond design (7.66 s). The `curve number and runoff` stage holds a failed outbound `GET` (red marker) that took 4.01 s.

The pond design in Figure 2 was labelled “confidence low: land cover or soil group assumed”. Its trace (Figure 5) shows the breakdown:

- delineating the catchment: 9 ms;
- fetching 45 years of daily rainfall: 1.65 s, of which 1.03 s is the Open-Meteo `GET`;
- the curve-number stage: 5.81 s, of which 4.01 s is one failed `GET`, marked as an error.

The failing span’s attributes, read from Jaeger, name the call and the reason:

```
url.full = https://rest.isric.org/soilgrids/v2.0/properties/query?lon=81.2992&lat=21.2481&...
error.type = URLError
otel.status_code = ERROR
otel.status_description = URLError: <urlopen error [Errno -5] No address associated with hostname>
event: exception (exception.type = urllib.error.URLError, with stack trace)
```

So the “low confidence” was not a modelling problem. The soil database’s *hostname did not resolve* for 4 s, the provider fell back to a stated default soil group, and the design said so. A later lookup from the same container resolved normally, so this was the campus network’s intermittent DNS, the same fault that reset Docker Hub pulls during setup. The trace also shows the cost: 4 s of a 7.7 s request was spent waiting on a name lookup, which argues for a shorter DNS/connect timeout on that provider.

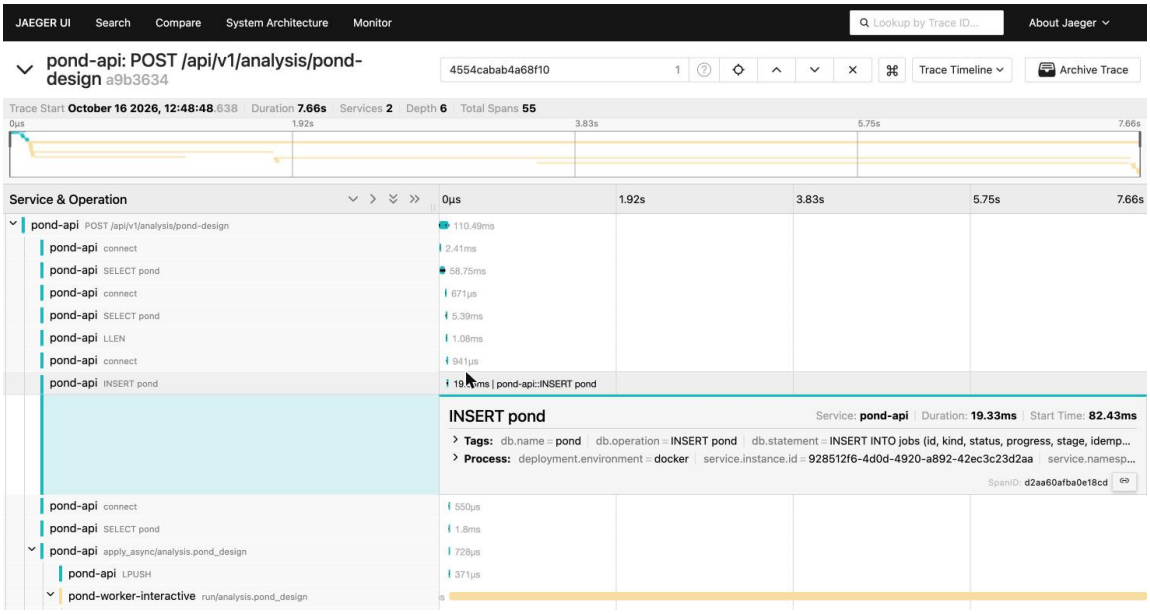


Fig. 6. Span detail for an automatically instrumented SQL statement (`INSERT pond`, 19 ms). The SQLAlchemy instrumentation records `db.system`, `db.name` and the statement; the Process block lists the Resource attributes that identify the container.

3.4 Logs, correlated to the trace

Figure 7 is the same incident seen from the log side. The warning has the *same trace id* as the trace in Figure 5. That is the practical value of correlation: starting from either a slow request or an error message, one click reaches the other half of the evidence. Grepping timestamps across three containers is no longer needed.

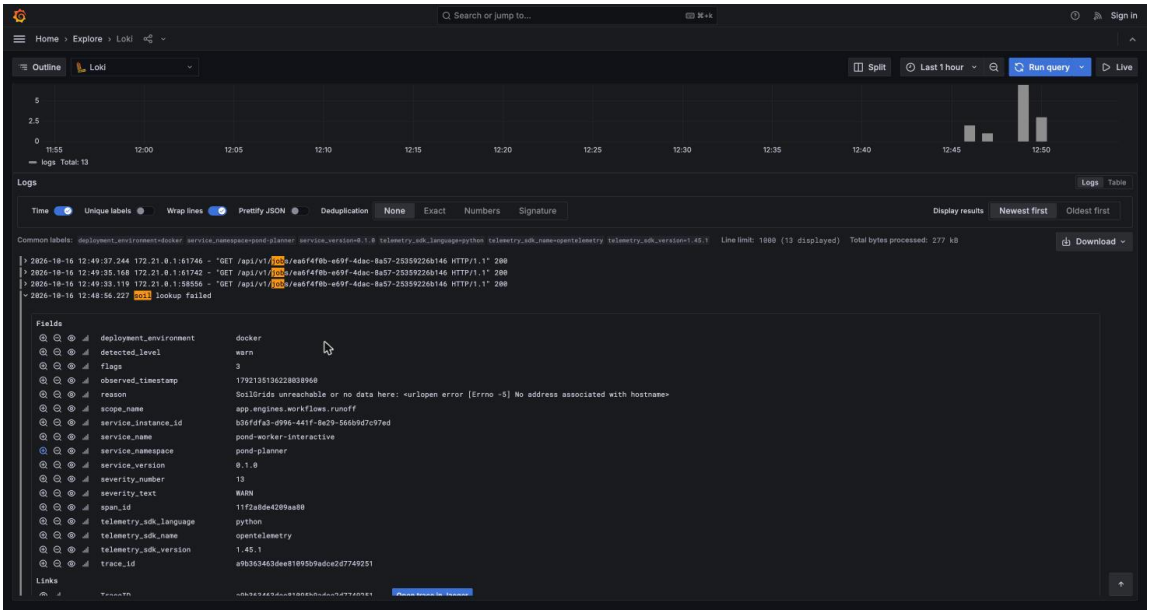


Fig. 7. Grafana Explore over Loki: the worker's `WARN soil lookup failed` record, exported over OTLP. It carries the reason, its service, `span_id` and `trace_id = a9b363463dee...`; the *Open trace in Jaeger* link goes to the trace in Figure 5.

3.5 Metrics

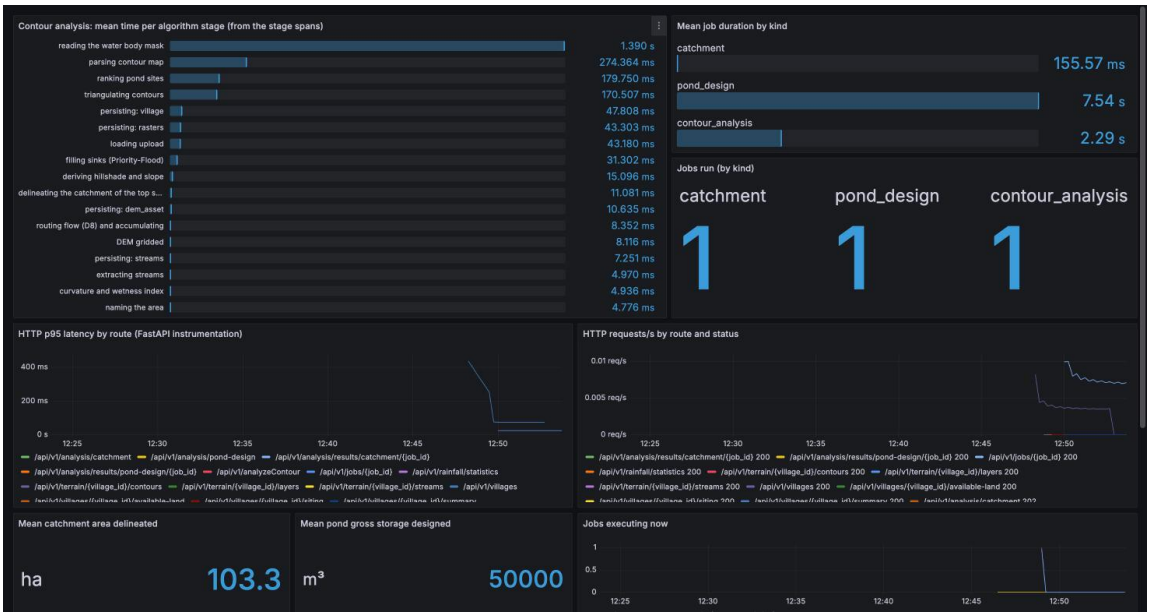


Fig. 8. Grafana dashboard *Pond — OpenTelemetry*, from the OTel metrics in Prometheus.

Figure 8 has six parts:

- **Top left:** mean time per pipeline stage, sorted. This is the `pond.job.stage.duration` histogram recorded as each stage span ends; the same finding as Figure 4, but aggregated over every run.
- **Top right:** mean job duration by kind (catchment 156 ms, contour analysis 2.29 s, pond design 7.54 s) and job counts.
- **Middle:** HTTP p95 latency and request rate per route, from the FastAPI instrumentation.
- **Bottom:** the domain metrics. The mean catchment delineated is 103.3 ha, the mean designed storage is 50 000 m^3, and a gauge counts jobs executing now.

Traces explain *one* request; metrics show the *trend*. For example, a regression in ranking time would show here before anyone opened a trace.

Behind the screenshots, the collector's debug exporter proves that all three pipelines are flowing (`docker logs pond-planner-otel-collector-1`):

```
info MetricsExporter {"data_type": "metrics", "resource metrics": 2, "metrics": 16, "data points": 49}
info LogsExporter {"data_type": "logs", "resource logs": 1, "log records": 2}
info TracesExporter {"data_type": "traces", "resource spans": 2, "spans": 11}
```

4 UNDERSTANDING: WHAT OPENTELEMETRY GIVES THIS SYSTEM

Monitoring. The dashboard answers "is it healthy and how fast" continuously: request rates and p95 latency per route, job durations by kind, jobs in flight, and the domain outputs themselves. A sudden fall in mean catchment area would flag a broken DEM source, not just a slow one.

Debugging. A trace turns "the design says low confidence" into "SoilGrids DNS failed after 4 s in this stage of this job". The error span carries the exception and stack trace, and the correlated log says what the code decided to do about it.

Performance analysis. Stage spans showed that network I/O, not hydrology, dominates the pipeline. They also show the queueing delay between API and worker, which is the number to watch when the bulkhead pools are resized.

Understanding the architecture. The trace *is* the sequence diagram of the asynchronous job design (ADR 0008): API, then Redis, then worker, then stages. It is generated from the running code, not drawn by hand, so it cannot go out of date.

Design choices and their reasons.

- **A collector rather than direct export.** It keeps backends out of the code, and it batches and limits memory in one place.
- **Explicit SDK setup rather than the zero-code agent.** The agent cannot see pipeline stages, and about 80 lines of setup can be explained line by line.
- **Batch processors.** Spans are exported on a background thread, so a request never waits on the collector. The cost is a few seconds before a trace is visible.
- **OTel metrics beside `prometheus_client`.** The OTel metrics run alongside the existing P6 metrics rather than replacing them, because the backpressure logic and the P6 dashboard depend on the old ones.

The full record is ADR 0022 in the repository.

Limits, stated.

- Reads of the Copernicus DEM go through GDAL's C library (libcurl), which Python instrumentation cannot see. They appear as stage time, not as HTTP spans.
- The Celery worker writes stdout in Celery's own format, so trace ids appear in its *exported* logs, not in its console.
- The lab VMs cannot run Docker, so the deployed replicas keep telemetry off by default.
- Sampling is 100 %, which is right for a demo and wrong for production volume, where a tail-sampling processor in the collector would keep only slow or failed traces.

Repository. https://github.com/Rahul5977/CSD-Course/tree/main/Project/1.PondPlanning — `app/core/telemetry.py`, `infra/otel/collector.yaml`, `infra/grafana/dashboards/pond-otel.json`, `tests/test_telemetry.py`, `docs/adr/0022-opentelem`

AI tool use. An AI coding assistant was used for the instrumentation, the configuration and this report. Every design decision above was reviewed and can be explained by the author.